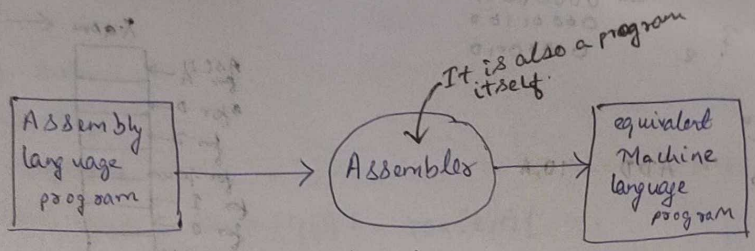
29/01/26



Context digram

- Identifiers :- User chosen names for data, instruction address, constants.
- Assembly language Instruction. eg: ADD A, X
- Machine instruction lang.

format - Few bits for op code
Then few bits for addressing modes of the operands.
Then bits for specifying the operands.
  - if operand is a register then No. of bits will be defined by no. of registers available.
  - if operand is memory address, No. of bits req. depend on size of memory.
  - if operand is an immediate operand, Actual value is given in instruction.
  - In indirect mode, the actual address is somewhere else, i.e indirectly given.

Codes used to indicate operation are called opcodes. In machine lang. they are called Numerical opcodes. & In assembly lang. they are called Mnemonic opcode

02/02/26

- Constructing the machine code of an Assembly instruc^n involve bit level operations. (Like shifting, OR etc)

Eg:-

To see man page of an item trick inside vi
Put cursor on the item & click shift+K
In command mode.

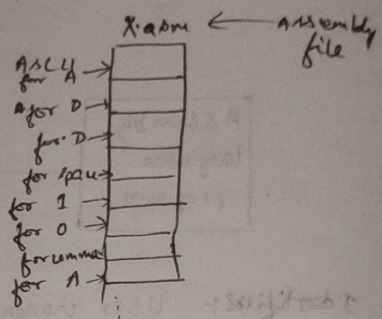
— 00000011
00001100
00000010

3 2

30

0011 0000

00111100 . ADD 10,A



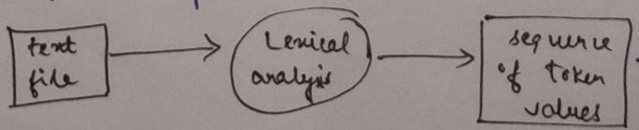
05/02/26

The first exercise the assembler has to perform is to recognize sequences of bytes that make sense, these sequences are referred as "tokens".

This exercise of recognizing the tokens is called lexical analysis.

- All user defined variables are termed as identifiers & all identifiers are categorised in single token.
- Each Mnemonic has separate token.

Here, the assembler reads tokens & give a number.



```
main()
{
    int token;
    FILE *ifp;
    ifp = fopen("  ", "r");      /* open the i/p assembly program */
    while ((token = lex.an(ifp)) != TOK_EOF){
        if (token == TOK_ADD){
            :
        }
        else if (token == TOK_COMMA){
            :
        }
            :
    }
    fclose(ifp);
}
```

if else ladder

⇒ I/p file to assembler -

```
        START  1000
        LOAD   X
AGAIN:  ADD    10,X
        CMP    A,B
        JNZ    AGAIN
        :
        JMP    NEXT
        :
NEXT:   ADD    10,Y
        :
X       DW     2
Y       DW     4
        :
        END
```

while converting each instruction to machine instruction,

NUMERIC OPCODE | ADDRESS OF X

NUMERIC OPCODE | ADDRESS OF AGAIN

Now, while going through instructions it may not be possible to generate machine instructions of all instructions at once, since, addresses of some symbols may not be known.

So, In First pass, we prepare a symbol table, then in further passes the complete machine instructions can be generated.

Symbol	ADDRESS
X	
AGAIN	
:	

Forward Analysis

(Forward reference)

* In the 2nd pass, the lexical analysis can be avoided, if in the 1st pass the sequence of tokens is saved.

09/02/26

5)

Lexical analyzer:

```
# include <token.h>

if (c= fgetc(ifp) == ',')
    return (TOK_COMMA);
else if (c == ':')
    return (TOK_COLON);
else if ( c >='0' && c <='9') {
    loop to read all digits & return
    a single token.
      return (TOK_NUM)
}
else if ( (c >= 'A' && c <='z') || (c >='a' && c <='z')
                    || (c == '_'))
    loop to read a word
```

Mnemonic
Identifier.

buffer to string

```
    if (strcmp(toktxt, "ADD") == 0)
        return (TOK_ADD)
    else if (strcmp(toktxt, "CMP") == 0)
        return (TOK_CMP)
    else
        return (TOK_ID)
}
```

Here, we are one by one comparing the toktxt with each Mnemonic, i.e Linear search,
We can improve it by using binary search.
(Sorting can be done during ~~initial scanning~~ creating the lexical analyzer only)
Not in runtime.

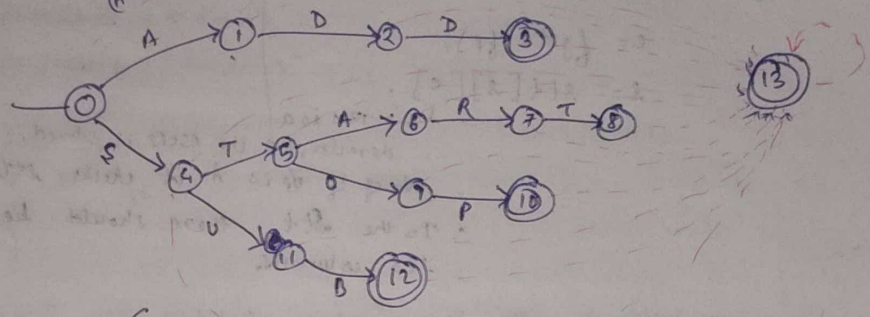
Better than these two is Finite Automata.

token.h

```
# define TOK_COMMA 7
# define TOK_COLON 8
```

Say: e.g.: ADD, START, STOP, SUB ← Mnemonics.

$(A-Z)-\{A,S\}$



State Transi^n Diagram

	A	B	C	D				S		
0	1	13	13	13	13	-	-	4		13
1	13	13	13	2	13					
2										
3										

states

stt[ ][ ]

State Transi^n Table

to get the next state from a current state we do stt[ ][ ] & directly get the next state. In this approach No searching is involved.

flex is a tool used to make a lexical analyzer on giving regular expressions

12/02/26

Regular expression For -

Numbers - $[0-9]^+$

Identifiers - $[a-z\ A-Z][a-z\ A-Z\ 0-9]^*$

• State is stored in just a variable.

say - int s = 0 ;

c = fgetc(fp);

s = stt[s][c] ;

↳ This is a character, but its ASCII is stored.

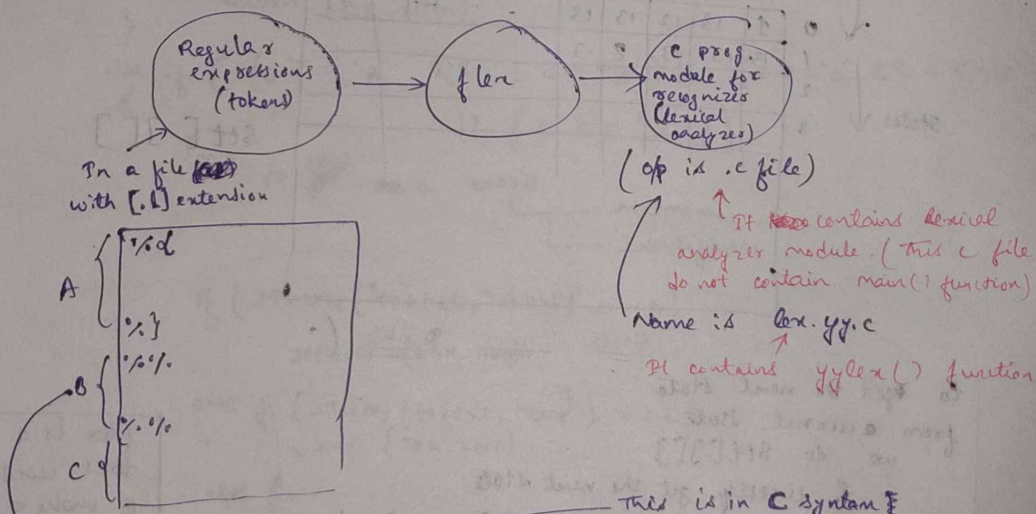
∴ if i/p is A it checks stt[0][65]

∴ In the stt, there should be 128 columns.

This making of STT is a very tidious job & error prone.

But it is highly efficient.

∴ A tool is used to create STT i.e FLEX



Action

More actions if req.

```
%%
ADD   {return(TOK_ADD); ...}
SUB   {return(TOK_SUB); ...}
START  ...

[0-9]+   return(TOK_NUM)
  :
  :
"\n"     line_num ++;
  :
%%
```

In secA, header files are included & variables are initialized.

```
%{
    #include <stdio.h>
    #include "token.h"
    :
    int line_num=0;
    FILE *ifp;
    :
%}
```

In secC, if task is small main & other functions req. can be written here.

```
%%
    int main()
    {
        int tok;
        yyin = fopen(ifp)
        while((tok = yylex()) ..
    }
```

file pointer

Also there is a standard input buffer

char yytext[]

This buffer stores the assembly program text while generating tokens, e.g. ADD, then 10, then X for ADD 10, X.

Date - 13/02/25

Note, tokens ⟶ Numeric opcodes.

Mnemonic Table

token	Type	opcode	No. of operands
ADD	3		
SUB	3		
CMP	3		
START	1		

Assembly program contain –

1. Imperative statements
2. Assembler directive
3. Declarative statements.

The 1st column is redundant, as it contains the tokens of Mnemonics which are equal to the row no. only.

ADD → 0
SUB → 1
CMP → 2

- Location counter processing –

Location counter is a variable counting the no. of bytes occupied in the memory while generating machine code

- During 1st pass, the token file is stored in memory. So, that 2nd pass can be done using the token file only, No need for running lexical analyzer again.

- Now, ~~during~~ In that token file, ~~[illegible]~~ many tokens will be same as all ~~constant~~ immediate values will have same token, all identifiers will have same token.

  ∴ There must be a distinction betw, them, so, either the string as a whole can be stored ~~just~~ after the token, like,

```
START 5000          25
ADD   10, X         '5'
JMP   NEXT          '0'
                    '0'
                    '0'
                    '0'
                    '1'
                    '8'
                    25
                    '1'
                    '0'
                    '0'
TOK-ID ---->        26
                    3 <---- Posin of X in symbol table.
```

IC (Intermediate code)

- Imperative statements are the ones which are actual instructions to be executed by CPU, & will be present in the code segment.

- Assembler directives are not for CPU, these are for assembler to do some works in advance, like

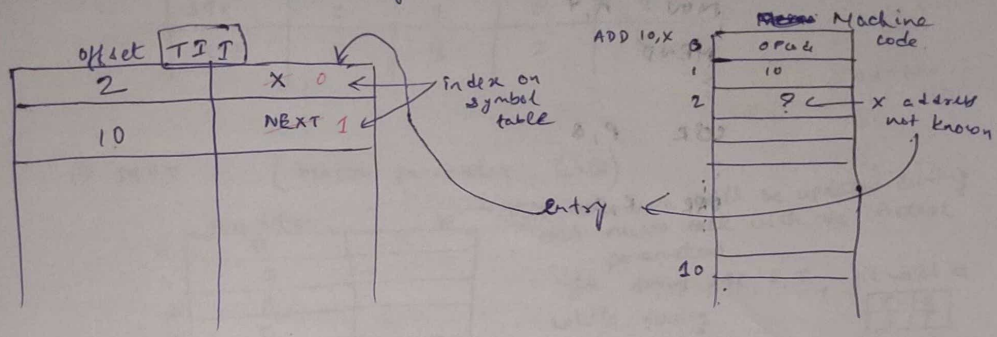
  ONE DC '1' <---- It (Assembler) checks the location counter value say, LC = 500, so Assemble put "00000001" at 500 (1)

This is the reason why DC, DS are put at the very end or at other separate data section, as if these are put in betw, the Assembler will store the constant value in some address inside the code segment, which ~~[illegible]~~ the CPU will try to execute as an instruction.

19-02/26

- **Single Pass Assembler :-**

During 1st pass we will be maintaining a table - Table of Incomplete Instructions (TII)



Symbol table

	Symbol	Addr	Type	Size
0	X	-1	-1	-1
1	NEXT	-1	-1	-1
	⋮			

More than two passes are also possible

- **Macro :-**

~~# define incr(4)~~

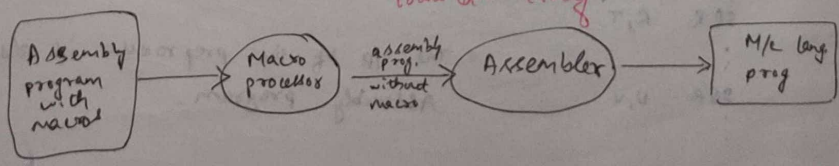
define incr4(x) $x = x+4$

Handled at translation time i.e ~~run~~ in run time there is no trace of calling the macro, the calling is already replaced by $x = x+4$ during translation itself.

```
#define TRUE 1
#define FALSE 0
```

Manifest constant

functions have overheads, i.e jumps etc



Eg: code for preprocessor:-

```
MACRO  SQR (X,Y)
MOV    X,A
MULT   X,A
MOV    A,Y
MEND
:
SQR    P,Q
:
SQR    R,S
```

23/02/26

Macro v/s subroutine.

- ~~In case of Macro~~, If multiple calls are there in the program, then it will increase the size of the program, more in case of Macro compared to subroutine, as each of the macro call will be replaced with the actual macro code, while in use of subroutine the subroutine is written once only, while there will be multiple jumps in this case.
- So efficiency will be more in case of macro, while size of program will be less in subroutine (if only multiple calls are there)
- Preprocessing involves lexical analysis.

  cc -E ... for preprocessing

- Parameters during macro definition are called Formal parameters while " " " calling " " Actual " .

Macro defn

```
MACRO  SQR &P,&Q
MOV    &P,&Q
MULT   &P,&Q
MEND
S1
S2
:
SQR    R,T
:
SQR    U,V
:
```

special symbol for macro.

Three cases where, the 1st item of an Assembly statement is an identifier is -

(i) Declarative statements (X DCW 10)

(ii) Labelling an instruction (NEXT ADD 10,X)

(iii) Macro calling statement.

* The o/p of the preprocessor is also an Assembly program.

Macro preprocessor

Data structures :-

① MNT :- (Macro Name Table)

Starting address of parameter at MPT

Parameter info

No. of entries MPT

Name				
SQR	0	2	0	2
XYZ	2	3	2	5

No. of entries in MDT.

Starting address of statements at MDT

② MPT :- (Macro parameter Table)

	Parameter	
0	P	R
1	Q	T
2	A	
3	B	
4	C	

This column will be updated during each macro call with the Actual parameters.

i.e during SQR R, T, it will be

P	R
Q	T

while during SQR U,V, it will be

P	U
Q	V

③ MDT :- (Macro Definition Table)

0	MOV P, Q
1	MULT P, Q
⋮	...

26/02/26

while running through the Macro defination & updating MNT, MPT, MDT, we can ease our future work (during Macro call) by not directly putting the parameters in MDT but the address or posin of those parameters in the MPT in MDT.

- Macro variables.
- Conditional Expansion.

MNT, MPT, MDT are useful only during preprocessing, i.e after preprocessing these are gone

```
MACRO    MSTCPY  &S, &D, &N
LEV      &I
SEV      &I, 0
AGAIN    MIF     (&I GE &N) OVER
         MOV     &S[&I], &D[&I]
         MINCR   &I, 1
         MGO     AGAIN
OVER     MEND.
.
.
.
MSTCPY   K, Q, 5
```

Greater than or equal to

Here, there is Conditional Expansion. Since, there is a check

Macro variable Table

MVT

&I	&N

The o/p Assembly program will contain —

```
MOV    K[0], Q[0]
MOV    K[1], Q[1]
.
MOV    K[4], Q[4]
```

- Parameter passing:-

printf ("%d", x) , Here, actual the value contained in x is passed, i.e pass by value.

scanf ("%d", &x) , Here, the address of x is passed i.e pass by reference.

27/02/26

* In C, there is no provision like pass by reference

Positional parameters:-

The associa^n of actual parameters to formal parameter is done according to their relative posi^n in the list of parameters.

Keyword parameters:-

When a macro is called the actual parameters are explicitly associated to the formal parameter irrespective of their relative positions.

E.g.:

```
MACRO   SQR   &P= , &Q=
  :
MEND
  :
SQR   &Q=R , &P=S
```

Default parameter can be passed here,

```
MACRO   MINCR   &P= , &Q=1
  :
MEND
  :
MINCR   &P=R
```

← default value.

← Only 1 parameter is mentioned, as the other parameter already has a default value.

```
#ifdef TRUE
}
#endif
```

→ This will be processed by preprocessor only if TRUE is defined.

Check what happens when keyword & positional parameters both are included (~~practice~~ order of parameters)

what is the use of "\" or "/" in Macro definition

```
#define DEBUG

#ifdef DEBUG
printf("x is %d\n",x)
#endif
```

02/03/26

SSCD

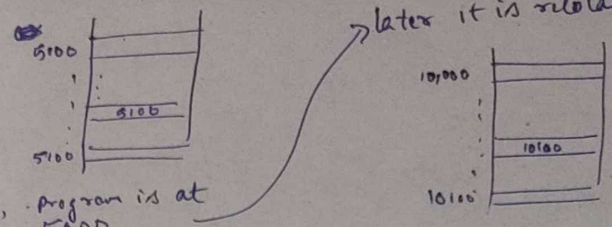
- **Loader :-**

A program which read from 2° storage & place it in memory.

communicate with OS for memory allotment.

Relocaⁿ :- Moving program from one locaⁿ to another.

E.g :-



later it is relocated to 10000

Initially, program is at 5000

Here Relocation factor is 10000 − 5000 = 5000, & Now all the addresses inside the program are updated by adding the Relocation factor.

* The assumed starting address of the program is only known to assembler, but in order to calculate the relocation factor, the relocator needs that.

The assumed starting address is also communicated to the relocator via the exe file only (containing the machine instruction),

∴ The Exe file not only contain machine instruction, it also contain extra info, i.e the Assumed starting address, & mostly in the starting (4 bytes).

One more thing the Exe file contain, is the list of positions in the machine code, where there is an address. ~~which we will be needy to be updated~~

NOT the absolute posⁿ but offsets

These locaⁿs are req. bcoz those addresses need to be updated.

The table is Relocation Table (RT), we need this for performing Relocation.

These extra part in exe file are included in Header part of Exe file

* ⇒ One extra thing is also req to be explicitly mentioned in exe file, i.e the size of the RT.

13/03/26

PROGRAM LINKING

The actual function definitions are in libraries which are linked with our prog. using linker.

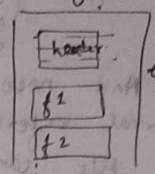
To let the compiler know that identifiers like printf, scanf, etc are actually functions.

header files contain the function declaration not function definition

declaraⁿ = int f1();
prototype = int f1(int n);

- Source files are ASCII files which can be opened using "vi".
- In C, source files extensions are .c, .h
- Library files in C in linux have extensions like .a, .so ← shared object. (.a → Archive)
- Output of translator (→ Assembler/Compiler) is not complete, few things are not present yet like the address to external function call (ie libraries) or addresses to global data objects
  → contains unresolved external references
- O/p of translator with unresolved external references are called as object files. (Extension in linux, in C prog. is .o; " windows is .obj)

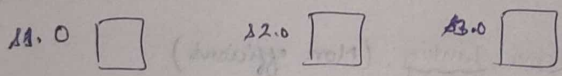
- .a files i.e Archive files contains ~~the contents~~ all the .o files combined which contain the library functions along with a header/catalog which specifies the posin of each func^n in that .a file at the beginning.

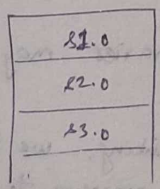


← .a file.

- Now a day there are multiple .c files created by us, which contains parts of our complete program. (Excluding library part for now).

Now, 1st ly all the .c files are compiled to get .o files & then all these .o files are combined into one final executable file a.out after two operations. i.e Relocation & Resolving unresolved external references.





a.out

Need to be done since each object file will assume the same starting address. Different reloca^n factor need to be added to each .o files such that when they are executed i.e put in RAM they do not superimpose on each other.

- Each .o file contain three tables generally -
  - (i) ~~Reloca^n~~ Reloca^n table containing the ~~add~~ posin^s where the relocation factor need to be added.
  - (ii) Linking table containing, two columns, one is offset & other is the name of the object (either code object (function) or data object (global data structure) which tells, at what posin^ in the file an object is located whose address is not known yet (i.e where to jump), Eg:- the file calls printf or scanf functions, now the linking table will contain the positions in the file where these are called, as these are the posin^s where we need to insert the address of the subroutine

iii) Some sort of ~~data~~ object table, which again contain two columns, one is the offset & the other is the name of the objects defined in the file.
(either use object or data object)

And now we know, in order to resolve those unresolved external references which are mentioned there in the linking table of each file can be resolved by scanning this last mentioned table of each file to get the address of the objects, since we will get the address where we need to jump.

- After these relocation & liking we get the executable file. it will have a single relocation table.

The linking we have done till now is called Static Linking.

Another one is Dynamic Linking. (More efficient).

How it is more efficient —

(i) Few modules eg:- libraries may be already loaded in memory (RAM)

(ii) In ~~dynamic~~ dynamic linking, we have the advantage of not loading all the modules of the program to run, those can be dynamically loaded & linked as and when req.